Homework 1 - Hadoop and Common-Followee Top-10

Reference implementation and report draft | Prepared 3 November 2026

Name: _____ SID: _____

Evidence status. The algorithms, scripts and local validation are supplied. No four-VM cloud cluster has been deployed in this session. Therefore the required cloud screenshots, four-VM WordCount performance table and a cloud-scale execution claim cannot be supplied yet. Values marked NOT MEASURED are intentionally unfilled. Local verification is identified separately in the accompanying results files.

The assignment deadline is 9 November 2026 at 23:59; its time zone is not specified in the PDF. The course submission system should be checked.

Authorship and acknowledgment. This draft and reference implementation were prepared with ChatGPT/Codex assistance. The assignment requires independently produced work, disclosure of discussions, and acknowledgment of external sources. The student must follow the course policy, produce the final submission accordingly, and sign the declaration only if it is accurate. No signature or originality assertion has been made here.

Deliverables and remaining evidence

Requirement	Supplied material	Remaining evidence
Q1(a), 10 marks	Single-node configuration and smoke-test commands	Real screenshots on the VMs
Q1(b), 20 marks	Four-node deployment procedure and paired experiment runner	Four-VM cluster and measured comparison
Q1(c), 20 marks	Four-configuration sweep and task timing collector	Actual Yelp runs and measured table
Q2(a), 50 marks	Code, full local-Hadoop results and all suffix exports	Required-cluster execution evidence; choose own suffix
Q2(b), 15 bonus	Same blocked implementation for the large graph	Large input, execution and suffix export

This report describes how the supplied implementation addresses each requirement. The detailed deployment commands are in `cluster/DEPLOYMENT.md`; the experiment commands are in `README.md`.

1. Q1(a): single-node Hadoop

The selected software stack is Ubuntu 22.04 x86_64, Java 11 and Hadoop 3.3.6. Hadoop 3.3.6 satisfies the assignment's version requirement of 2.7.1 or above. The old Ubuntu/Java versions in the tutorial are examples rather than assignment requirements. Java compatibility is documented by Apache [1].

`bootstrap_node.sh` installs Java, SSH, Python and Hadoop under a non-root VM user. It verifies the Hadoop archive against Apache's SHA-512 checksum. `configure.py --mode single` generates the four site XML files and a workers file containing only that VM. This follows Apache's single-node setup model [2].

The single-node configuration uses HDFS replication 1, YARN as the MapReduce framework, local directories below `~/hadoop-single`, and a JobHistoryServer. This setup is to be verified separately on each VM before changing to the four-node configuration.

For a fresh, empty namespace, format the NameNode once and start HDFS, YARN and the history server. An existing NameNode must not be reformatted. Upload the following two-line smoke-test input:

```
hello hadoop
hello mapreduce
```

The built-in Hadoop WordCount example must return the following counts, irrespective of output-file order:

```
hadoop     1
hello      2
mapreduce  1
```

Required evidence: actual Hadoop and Java version output, the successful job and result, and `jps` showing the single-node services. The above counts are an analytically known smoke-test expectation, not a fabricated screenshot or benchmark result.

Stop the single-node daemons on every VM before moving to the multi-node setup. Retain its screenshots and keep its storage separate from the new multi-node namespace.

2. Q1(b): four-VM setup

The intended topology has one master and three workers. This separates cluster coordination from data and task execution [3].

VM	Hadoop services
master	NameNode, SecondaryNameNode, ResourceManager, JobHistoryServer
worker1	DataNode, NodeManager
worker2	DataNode, NodeManager
worker3	DataNode, NodeManager

A practical starting configuration is 4 vCPUs and 8 GiB RAM per VM. Worker YARN capacity is set to 6,144 MiB and 4 vcores. Task containers request 1,536 MiB with a 768 MiB Java heap. These are proposed settings, not descriptions of provisioned hardware. The actual VM type and storage must be recorded after deployment.

All VMs should share a private network. Private hostnames must resolve consistently. The Hadoop user needs passwordless SSH from the master to the workers. Hadoop Web and RPC endpoints should remain private; SSH forwarding provides access to the Web interfaces.

`configure.py --mode multi` sets the HDFS endpoint to `hdfs://master:9000`, replication to 3, ResourceManager to master, history interfaces to ports 10020/19888, and the workers list to exactly worker1-worker3. Separate `~/hadoop-multi` directories avoid reusing a single-node namespace.

After copying the generated files to all VMs, format the new NameNode only once. Start HDFS and YARN, create the history directories, and start the history server. Verify the following before measuring performance:

Check	Required observation
`hdfs dfsadmin -report`	3 live DataNodes
`yarn node -list`	3 RUNNING NodeManagers
`jps` on master	Coordination services, no DataNode/NodeManager
`jps` on workers	DataNode and NodeManager on each
WordCount smoke test	Correct word frequencies, successful job

Evidence not yet collected: cloud console VM list, service/process screenshots, HDFS/YARN status, and the successful four-VM smoke test.

3. Q1(b): small files versus one merged file

The exact Shakespeare archive specified in assignment reference [11] has been downloaded and prepared. It contains 196 text files; the normalized merged input is 6,460,189 bytes. Both equivalent inputs and a per-file checksum manifest are supplied in `data/shakespeare_prepared`. `prepare_words.py` adds a terminating newline to any nonempty file that lacks one, applies the same normalization to both inputs, and concatenates the normalized files. This prevents words at file boundaries from being accidentally joined.

As a local correctness check, both inputs were executed with real Hadoop LocalJobRunner. They produced identical frequencies for 72,379 distinct tokens, with 196 and 1 map tasks respectively. These are not four-VM timing measurements. The canonical output and checksum are included in `results/shakespeare_validation.json`.

For the assessed benchmark, use the same built-in WordCount program, cluster settings and number of reducers for both inputs. Run each condition three times, alternating their order to reduce a consistent first-run advantage. Do not execute competing jobs during the comparison. The output frequencies must agree: the runner sorts each output and compares SHA-256 hashes.

Setting	Actual maps	Reducers	Job times, 3 repetitions	Output equality
Many small files	NOT MEASURED	1 planned	NOT MEASURED	NOT CHECKED
One merged file	NOT MEASURED	1 planned	NOT MEASURED	NOT CHECKED

Expected mechanism, not an observed result: with default TextInputFormat, many small files can create many short map tasks. Their initialization, scheduling, metadata access and per-task I/O may dominate useful work. Merging can reduce this overhead, as discussed by the course tutorial [4].

The merged condition is not guaranteed to be faster. If the merged Shakespeare text produces only one split, it may use only one mapper while the small-file condition exposes more parallelism. Runtime depends on the balance between launch overhead and useful parallel work, as well as caching, input size and I/O locality.

For the completed analysis, report both actual mapper counts and measured job times, then calculate `speedup = median(small-file job time) / median(merged-file job time)`. A value below 1 means the merged condition was slower. Explain the measured direction using task counts, task durations and counters rather than assuming the expected mechanism won.

This comparison measures default small-file behavior. A CombineFileInputFormat experiment would be a separate mitigation experiment and should not silently replace this baseline.

4. Q1(c): mapper/reducer experiment and timing table

Use the same actual Yelp dataset for every run [5]. The supplied procedure treats one uncompressed Yelp review JSONL file as text. If the course expects review text only, extract the `text` field consistently first. Record the precise input filename, byte count, SHA-256, preprocessing and HDFS block size. A gzip text file is unsuitable for this split-control experiment because ordinary gzip input is not splittable.

Target four combinations: (6 mappers, 1 reducer), (6, 3), (12, 1), and (12, 3). The runner sets both input split bounds to `ceil(input_bytes / target_maps)`. Actual mapper count is governed by InputFormat and split construction; `mapreduce.job.maps` is not a reliable way to force it. The script records actual counts and rejects a sweep yielding fewer than four distinct actual combinations.

Target M/R	Actual M/R	Map max/min/avg (s)	Reduce max/min/avg (s)	Job (s)
6 / 1	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT MEASURED
6 / 3	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT MEASURED
12 / 1	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT MEASURED
12 / 3	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT MEASURED

This compact table lists the four planned settings; expand it to one row per actual job, including its job ID (twelve rows for three repetitions). It groups the seven required time statistics. `timings.csv` writes each of the seven as a separate column, together with actual M/R counts and job IDs. Repeat each configuration three times; retain every run rather than reporting only the best result.

`metrics.py` reads completed jobs from JobHistory REST [6]. Job time is `(job.finishTime - job.startTime)/1000`. Task time is `(task.finishTime - task.startTime)/1000`; maximum, minimum and arithmetic mean are calculated over completed tasks of that type. Reducer task time includes shuffle, merge/sort and reduce. It is not only time spent inside the user's reduce function. Missing tasks or unsuccessful jobs are rejected. Raw job, task and counter JSON is retained.

More maps may improve parallelism but increase startup, sort/spill and scheduling overhead. More reducers may relieve a reduce bottleneck but increase shuffle fan-out and output fragmentation. Interpret the actual measurements using map/reduce duration dispersion, spilled records, shuffle bytes and failed-attempt counts. Job time is not the sum of task times because tasks overlap.

5. Q2: graph interpretation and exact ranking

For user u, let F(u) be the set of users that u follows. The required similarity is `s(u,v) = |F(u) intersection F(v)|`, excluding u = v. This is common-followee similarity: it must not be accidentally implemented as common-follower similarity or replaced by Jaccard, cosine similarity or a graph clustering method.

For each followee f, define L(f) as its distinct followers. Every unordered pair from L(f) contributes one to that pair's similarity. Summing these contributions over all followees gives the required intersection count.

Observed course input. The actual Dropbox file at assignment reference [7] was downloaded successfully. It contains a two-column numeric edge list with 2,420,766 nonblank edge lines, 1,768,149 distinct directed edges and 81,306 distinct endpoint IDs. There are 652,617 duplicate edge lines beyond the unique directed edges. Its compressed size is 10,621,918 bytes. This differs from the assignment prose describing about 2.5 million users. The count was obtained by deduplicating ordered edges with a disk-backed SQLite primary key and counting the union of endpoints. It is not an estimate.

The file's compressed SHA-256 is:

`d9f99b0e6a53b9204b8c215f41b3c10fb99a1e1e783858c012b06d0d3d4bd129`

The parser also accepts the assignment's `followee:followers` representation. For edge lists, the first column is assumed to be the follower unless `--edge-order followee-follower` is specified. The SNAP readme defines a Twitter/Gplus edge a b as a following b [9], supporting this convention. Format auto-detection alone cannot establish direction; a different course file would need its own direction check. The observed node/unique-edge counts agree with the public SNAP Twitter description [7], but no alternate download was substituted for the course URL.

All endpoints and empty adjacency-list heads are included in the user universe. Duplicate edges are removed before counting. Numeric IDs use Python arbitrary-precision integers. Ties are broken by increasing numeric ID, and a user never recommends itself.

The assignment does not specify how to resolve ties or users with fewer than ten positive-similarity candidates. The explicit convention used here is to fill remaining positions with zero-similarity users in increasing ID order, producing `min(10, N-1)` distinct candidates per user. This policy should be stated in the final report and checked against any course clarification.

6. Q2: six MapReduce stages

Job	Mapper/reducer operation	Output
01	Normalize edges, retain users, deduplicate composite edge keys	Unique edges and user markers
02	Keep local then global smallest K+1 user IDs	Tiny universe side file
03	Group by followee, spool followers into B-user blocks	Block payloads with block counts
04	Replicate blocks to block-pair keys and enumerate pairs	One unordered-pair contribution per shared followee
05	Sum contributions by unordered user pair	Exact positive similarity scores
06	Emit scores in both directions, secondary-sort and take K	All-user Top-10 records

Stage 01 keys are `(E, followee, follower)` or `(U, user, marker)`. The dedup reducer is also safe as a combiner: repeated application produces the same unique records. User markers ensure that users without positive-similarity partners are retained.

For jobs 01 and 05, `stream.num.reduce.output.key.fields=3` is explicitly set so a Streaming combiner preserves the full composite key. Omitting it can corrupt merge ordering across spills and invalidate deduplication or pair summation.

Stage 03 partitions on `(kind, followee)` and uses the full composite key for sorting. One followee's follower blocks are spooled to a temporary disk file. Its full follower list is not retained in reducer memory. Once the block count is known, records are emitted with the count required by the next stage.

Stage 04 routes block i to every unordered block-pair `(min(i,j), max(i,j))`. A diagonal pair receives one block; an off-diagonal pair receives left and right blocks. The partition key is `(P, followee, left_block, right_block)` and the side is an additional sorting field. A reducer processes at most two B-user blocks at a time. Diagonal groups emit combinations within a block; off-diagonal groups emit the Cartesian product. This covers every follower pair once for that followee.

Stage 05 partitions on the full unordered-pair key and sums integer contributions. Its sum combiner reduces repeated pair traffic where possible without changing the result. It never combines incomplete rankings or drops candidates before their total score is known.

Stage 06 emits `(u, -score, v)` and `(v, -score, u)` as composite keys. KeyFieldBasedPartitioner uses only u; KeyFieldBasedComparator sorts numerically by u, negative score and candidate ID. Thus all candidates for one user reach the same reducer in the required order [8]. The reducer retains only K positive candidates and fills any missing positions using the tiny universe file. Final records have the specified `user:id id ...` format without score fields.

7. Q2: correctness and scaling analysis

Similarity correctness. After deduplication, an edge appears once. For a fixed followee f, distinct followers are divided into disjoint blocks. Every distinct follower pair lies either within exactly one block or across exactly one unordered pair of blocks. Stage 04 therefore emits exactly one contribution for that pair and f. Stage 05 sums one contribution for every common followee, giving `|F(u) intersection F(v)|`.

Top-K correctness. Each positive score is sent to both endpoints. All records for a user are co-partitioned and sorted by decreasing score and then increasing candidate ID. The first K positive candidates are therefore the exact positive-score ranking prefix. No candidate is truncated before pairwise summation.

Zero padding and every-user correctness. Each user has a marker even when it has no positive candidates. If p < K positive candidates were retained, the globally smallest K+1 IDs contain at least K-p IDs other than the user and those p candidates, unless the entire graph has fewer than K+1 users. In that case they contain all users. Selecting unused IDs in increasing order therefore supplies exactly the correct zero-score suffix, without an N-by-N zero-pair expansion.

Let E be the number of unique edges, d_f the follower count of f, and `C = sum_f d_f*(d_f-1)/2`. Stage 04 emits C pair contributions. Its CPU/output work is O(C), and exact pair enumeration can be expensive even when the input itself is small. For the observed medium file, assuming first-column follower semantics, the maximum d_f is 3,383 and C is **154,933,408**.

With `k_f = ceil(d_f/B)`, the block-replication stage moves `sum_f d_f*k_f` follower-ID occurrences. Smaller B distributes heavy-followee work into more, smaller groups, but increases replication overhead. Pair reducers use O(B) IDs per group; block construction also uses O(B) buffered IDs plus O(d_f) temporary disk storage for the active followee. Parsing an original adjacency line additionally requires memory proportional to that line's length. Top-K reducers use O(K) candidate memory. Shuffle/sort storage remains external to these Python state bounds.

Map-output compression is enabled. This compresses shuffle/spill output, not all HDFS stage files. Input/output checks, available-disk monitoring and Hadoop counters remain necessary. `STOP_AFTER_BLOCKS=1` allows an explicit capacity checkpoint before pair enumeration; `RESUME_AFTER_BLOCKS=1` continues from it. Blocking improves memory bounds and load distribution but does not guarantee that arbitrary graphs fit small VMs or finish within a fixed time.

8. Validation, suffix output and completion checklist

The automated semantic tests compare the blocked pipeline against an independent brute-force implementation based on followee-set intersections. They include the assignment figure, duplicate edges and input lines, empty input, an isolated user, followee-only users, zero-score padding, ties, leading-zero IDs, IDs larger than 64 bits, both directed edge conventions, explicit undirected mode and a high-degree blocked case. An additional 100 fixed-seed random graphs are checked. Timing-parser fixtures are explicitly synthetic and are not experimental measurements.

Completed medium-data execution. All six jobs succeeded using Hadoop 3.3.6 and Java 11 with LocalJobRunner and the local filesystem. The full result contains 81,306 users with ten distinct, non-self candidates each. All-user coverage and formatting were checked, and 1,000 fixed-seed sampled rankings were independently recomputed from deduplicated followee sets: zero mismatches. Files for all 1,000 possible student-ID suffixes are supplied. This validates the results and implementation, but is not four-VM/YARN or IE DIC execution evidence.

The full-result SHA-256 is:

`1d86de53417447e7a124c0cb9983b2761c834125e9e4620802c9d65a1b655ed9`

Using A=1, B=2, C=3, D=4 and E=5, the assignment example yields:

```
1:2 3 5 4
2:1 3 4 5
3:1 2 4 5
4:2 1 3 5
5:1 2 3 4
```

For example, users 1 and 2 share followees 3 and 5, giving similarity 2. Their similarity to other users is lower. With only five users, each line contains four candidates; the zero-score suffix follows the stated tie policy.

The complete result is computed for every user before submission filtering. `filter_suffix.py --suffix XYZ` retains users satisfying `int(user_id) % 1000 == int(XYZ)`. For suffix 007, this correctly includes IDs ending in 007. Filtering the graph before similarity calculation would discard candidate users or shared followees and is not equivalent.

To complete the assessed submission: obtain real four-VM deployment screenshots; run the Shakespeare comparison; run and export the four-configuration Yelp sweep; rerun the supplied medium graph code on the required course cluster if necessary; select the already supplied student-suffix subset; and optionally run the large graph for the bonus. Populate every NOT MEASURED entry using the preserved job IDs and raw records. Inspect input, all-user and suffix-result counts and include relevant output excerpts/screenshots.

No measured cloud speedup, cloud execution time, or bonus completion is claimed. The supplied medium all-user results and all 1,000 suffix subsets are from LocalJobRunner. They do not establish a four-VM or IE DIC execution. The `results/` directory preserves the validation records and successful job logs.

References and source acknowledgment

The uploaded Homework 1 PDF is the source of the question, marking scheme, output format and deadline. The supplied implementation, analysis and report use ChatGPT/Codex assistance as stated on the cover. The final submission should separately list actual human discussions and any additional sources used by the student.

[1] Apache Software Foundation. Hadoop Java Versions. https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=421957808

[2] Apache Hadoop 3.3.6. Single Node Setup. https://hadoop.apache.org/docs/r3.3.6/hadoop-project-dist/hadoop-common/SingleCluster.html

[3] Apache Hadoop 3.3.6. Cluster Setup. https://hadoop.apache.org/docs/r3.3.6/hadoop-project-dist/hadoop-common/ClusterSetup.html

[4] Course Tutorial 2. Single-node Hadoop, WordCount, and secondary sort. https://xuhappy.github.io/courses/BigData/tutorial/tutorial2/

[5] Yelp Open Dataset. https://www.yelp.com/dataset

[6] Apache Hadoop 3.3.6. MapReduce History Server REST APIs. https://hadoop.apache.org/docs/r3.3.6/hadoop-mapreduce-client/hadoop-mapreduce-client-hs/HistoryServerRest.html

[7] Stanford SNAP. Social circles: Twitter. https://snap.stanford.edu/data/ego-Twitter.html . Used to cross-check the observed course-file node/unique-edge counts, not as a replacement download.

[8] Apache Hadoop 3.3.6. Hadoop Streaming. https://hadoop.apache.org/docs/r3.3.6/hadoop-streaming/HadoopStreaming.html

[9] Stanford SNAP. Ego-network dataset readme (edge-direction convention). https://snap.stanford.edu/data/readme-Ego.txt

The exact course dataset URLs and Shakespeare URL remain those printed in the assignment. Dataset orientation and any changed course distribution should be confirmed with the instructor rather than inferred from a filename.

Student declaration - to be completed personally

The following is the declaration required by the uploaded assignment. It is included as an unsigned form. The student must ensure that it accurately describes the final work and that the course's independent-work and source-disclosure requirements have been met before signing. This draft is explicitly AI-assisted.

I declare that the assignment submitted is original except for source material explicitly acknowledged, and that the same or related material has not been previously submitted for another course. I also acknowledge that I am aware of University policy and regulations on honesty in academic work, and of the disciplinary guidelines and procedures applicable to breaches of such policy and regulations.

Signed (Student): _____

Date: _____

Name: _____

SID: _____

Names of people actually consulted or worked with:

_____

External material and AI assistance actually used, subject to course policy:

_____

_____